

人工智能与计算机学院

School of Artificial Intelligence & Computer Science

Project-3 Report:

Alarm System Based on Temperature Thresholds

COURSE TITLE : Curriculum Design for Embedded System & Interface Technology

STUDENT NAME : KHAN MD SHAKIL

STUDENT NUMBER : 2330130220

SUPERVISOR : 袁佳祺

ACADEMIC YEAR : 2025-2026

OBJECTIVE **3**

EQUIPMENT AND MATERIALS **3**

THEORY **4**

Temperature Monitoring: 4
Threshold Comparison: 4
Alarm Activation: 4

PROCEDURE **4**

Step 1: System Initialization 4
Step 2: ADC Configuration 5
Step 3: Threshold Setup 5
Step 4: Alarm Activation 5
Step 5: Testing and Debugging 5

CODE IMPLEMENTATION **8**

Threshold Definition 8
Main Logic in main.c 8

OBSERVATIONS AND RESULTS **10**

Observations 10

RESULTS **11**

CHALLENGES AND SOLUTIONS **11**

Challenge 1: Inaccurate Temperature Conversion 11
Challenge 2: Debouncing the LED Alarm 11

CONCLUSION **11**

Key outcomes: 11
Future extensions of this project could include: 12

REFERENCES ..... **ERROR! BOOKMARK NOT DEFINED.**

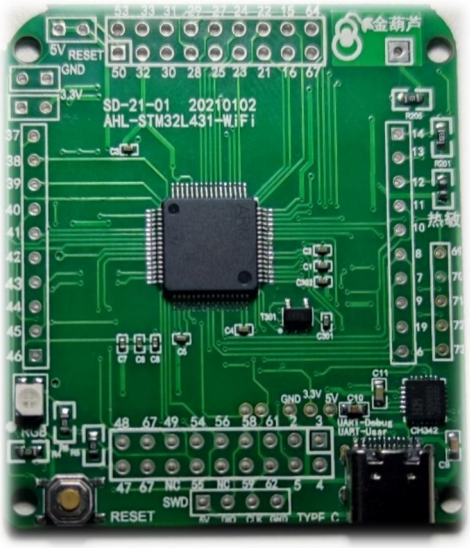
Objective

This project aims at developing an **Alarm System Based on Temperature Thresholds** using STM32 microcontroller. The main idea is to have a thermometer with a sensor that will check the temperature of the surrounding and display an alarm if the temperature is too high or too low. For the visual alarm, this project uses a red LED that is on as soon as the temperature goes beyond the critical level.

This system is intended to mimic realistic conditions under which temperature plays a critical role in control, for instance in industrial applications, medical equipment, and environmental control. Analog to digital conversion for temperature sensing is incorporated within the project along with GPIO pins for LED control and proper control logic to process and regulate temperature data in real time.

Developing this system, we show how and why it is possible to integrate microcontroller peripherals to create an effective and efficient embedded system that can be used in a range of contexts.

Equipment and Materials



- **Microcontroller**: STM32L431RC
- **Temperature Sensor**: Onboard temperature sensor
- **LED Indicators**:
    - Red LED (Visual Alarm)
- **Development Environment**: AHL-GEC-IDE
- **Additional Components**:
    - Breadboard and jumper wires for prototyping
    - USB cable for programming and UART communication

Theory

A temperature threshold-based alarm system is commonly employed for safety and protection against risks of high or low temperatures.

Temperature Monitoring:

- Analog output in terms of voltage is given by the temperature sensor for the measured temperature.
- The ADC peripheral of the STM32 microcontroller Apr grips this analog signal and convert it to a digital form.

Threshold Comparison:

- Standard temperature ranges for example the maximum safe temperature are employed as the reference.
- When the indicated temperature exceeds these thresholds an alarm sound is activated.

Alarm Activation:

- If the temperature goes beyond the upper limit, then the red LED is illuminated to give the warning that the temperature has gone high.
- The LED is switched off by the alarm system as soon as the temperature goes back to normal range.

This project simplifies the implementation by focusing on a single threshold: For the high temperature conditions, the red LED activation is used.

Procedure

Step 1: System Initialization

1. Set the clock to accurate time so that the system has the right timing.

2. Setup GPIO control for the red LED.
3. Configure the ADC to convert the temperature signal in analog form to a digital signal.
4. We have to configure UART for debugging and for logging the temperature values.

Step 2: ADC Configuration

1. Choose the ADC channel which is interfaced with the temperature sensor They are connected to the microcontroller through ports.
2. Set the ADC resolution to 12-bit in order to get the more precise results.
3. Switch on continuous conversion mode to be able to monitor temperature changes real time.

Step 3: Threshold Setup

1. Determine the high-temperature limit in the light of the functions the application is to serve.
2. This threshold value should be used as constants in the code for better understanding and to avoid future changes.

Step 4: Alarm Activation

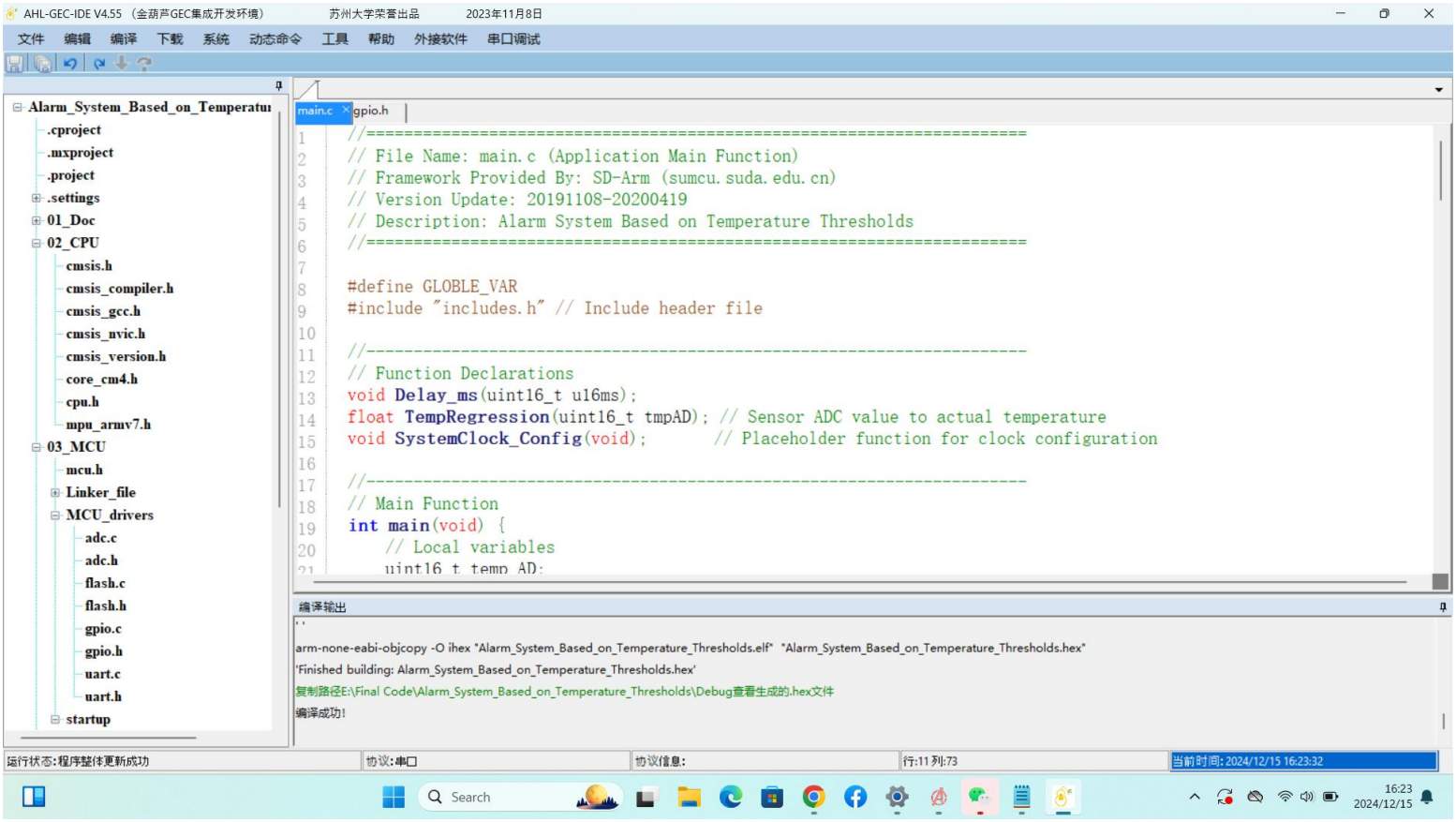
1. Keep on reading the ADC value and then convert into the temperature.
2. Check the current temperature with the predefined high temperate threshold.
3. If the threshold is crossed then turn on the red LED as a visual warning.
4. You should turn off the red LED when the temperature becomes normal.

Step 5: Testing and Debugging

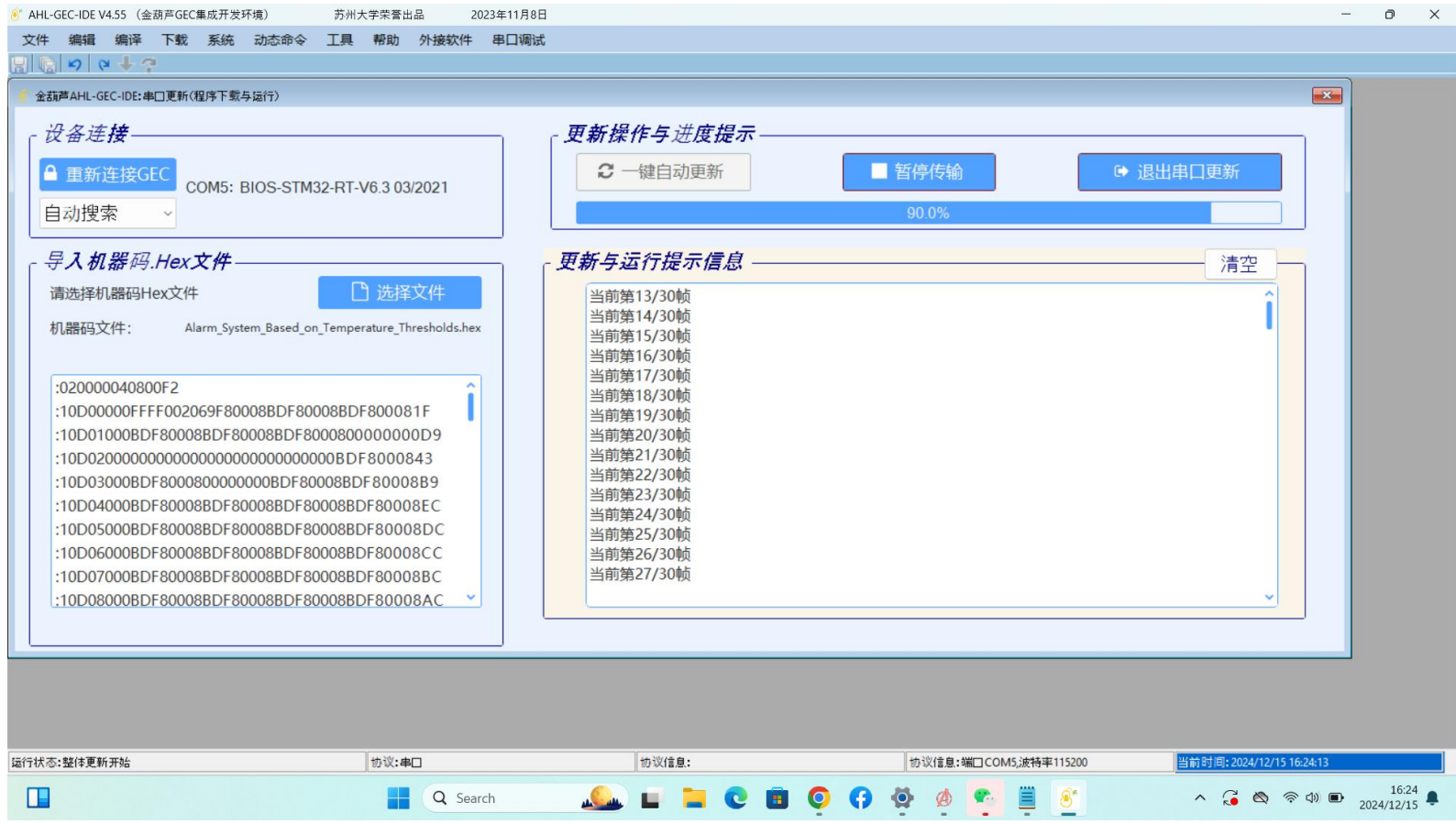
1. To display real time temperature data and the alarm status on the UART.

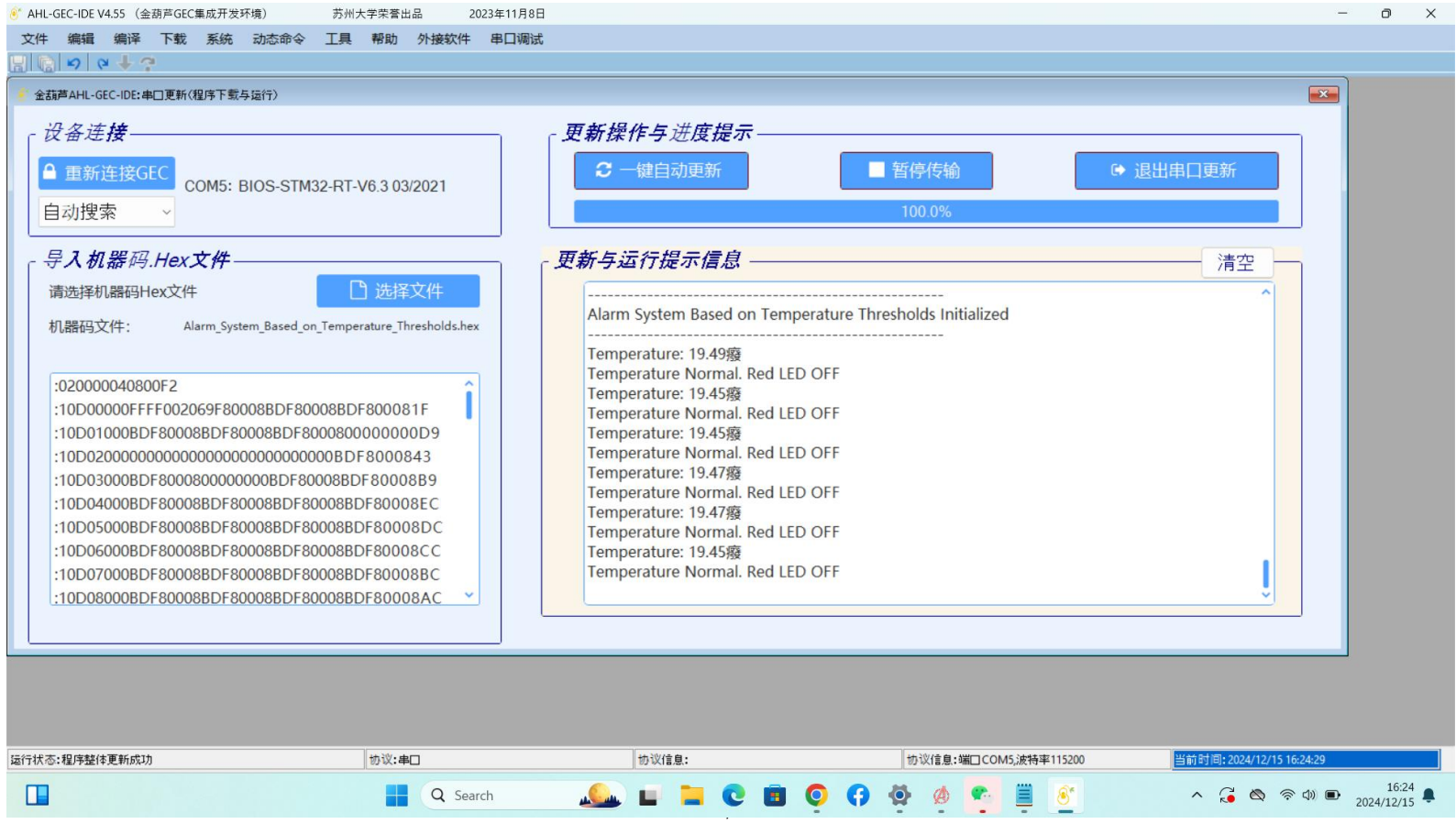
2. Apply heat to the battery to check the LED alarm system under high temperature conditions.

3. Observe the first and last LED, its activation and deactivation within the first



and second temperature range.


AHL-GEC-IDE V4.55 （金葫芦GEC集成开发环境） 苏州大学荣誉出品 2023年11月8日
文件 编辑 编译 下载 系统 动态命令 工具 帮助 外接软件 串口调试
金葫芦AHL-GEC-IDE:串口更新(程序下载与运行)
设备连接
重新连接GEC
COM5: BIOS-STM32-RT-V6.3 03/2021
自动搜索
更新操作与进度提示
一键自动更新
暂停传输
退出串口更新
90.0%
导入机器码.Hex文件
请选择机器码Hex文件
选择文件
机器码文件: Alarm_System_Based_on_Temperature_Thresholds.hex
:020000040800F2
:10D00000FFFF002069F80008BDF80008BDF800081F
:10D01000BDF80008BDF80008BDF80008000000000D9
:10D020000000000000000000000000000BDF8000843
:10D03000BDF80008000000000BDF80008BDF80008B9
:10D04000BDF80008BDF80008BDF80008BDF80008EC
:10D05000BDF80008BDF80008BDF80008BDF80008DC
:10D06000BDF80008BDF80008BDF80008BDF80008CC
:10D07000BDF80008BDF80008BDF80008BDF80008BC
:10D08000BDF80008BDF80008BDF80008BDF80008AC
更新与运行提示信息
清空
当前第13/30帧
当前第14/30帧
当前第15/30帧
当前第16/30帧
当前第17/30帧
当前第18/30帧
当前第19/30帧
当前第20/30帧
当前第21/30帧
当前第22/30帧
当前第23/30帧
当前第24/30帧
当前第25/30帧
当前第26/30帧
当前第27/30帧
运行状态:整体更新开始
协议:串口
协议信息:
协议信息:端口COM5,波特率115200
当前时间: 2024/12/15 16:24:13
Search
16:24
2024/12/15


AHL-GEC-IDE V4.55 （金葫芦GEC集成开发环境） 苏州大学荣誉出品 2023年11月8日
文件 编辑 编译 下载 系统 动态命令 工具 帮助 外接软件 串口调试
金葫芦AHL-GEC-IDE:串口更新(程序下载与运行)
设备连接
重新连接GEC
COM5: BIOS-STM32-RT-V6.3 03/2021
自动搜索
更新操作与进度提示
一键自动更新
暂停传输
退出串口更新
100.0%
导入机器码.Hex文件
请选择机器码Hex文件
选择文件
机器码文件: Alarm_System_Based_on_Temperature_Thresholds.hex
:020000040800F2
:10D00000FFFF002069F80008BDF80008BDF800081F
:10D01000BDF80008BDF80008BDF80008000000000D9
:10D020000000000000000000000000000BDF8000843
:10D03000BDF80008000000000BDF80008BDF80008B9
:10D04000BDF80008BDF80008BDF80008BDF80008EC
:10D05000BDF80008BDF80008BDF80008BDF80008DC
:10D06000BDF80008BDF80008BDF80008BDF80008CC
:10D07000BDF80008BDF80008BDF80008BDF80008BC
:10D08000BDF80008BDF80008BDF80008BDF80008AC
更新与运行提示信息
清空
Alarm System Based on Temperature Thresholds Initialized
Temperature: 19.49癈
Temperature Normal. Red LED OFF
Temperature: 19.45癈
Temperature Normal. Red LED OFF
Temperature: 19.45癈
Temperature Normal. Red LED OFF
Temperature: 19.47癈
Temperature Normal. Red LED OFF
Temperature: 19.47癈
Temperature Normal. Red LED OFF
Temperature: 19.45癈
Temperature Normal. Red LED OFF
运行状态:程序整体更新成功
协议:串口
协议信息:
协议信息:端口COM5,波特率115200
当前时间: 2024/12/15 16:24:29
Search
16:24
2024/12/15

Code Implementation

Threshold Definition

```
#define TEMP_HIGH 20   // High temperature threshold in degrees Celsius
```

Main Logic in main.c

```
#define GLOBLE_VAR
#include "includes.h" // Include header file

// Function Declarations
void Delay_ms(uint16_t u16ms);
float TempRegression(uint16_t tmpAD);
void SystemClock_Config(void);

// Main Function
int main(void) {
    // Local variables
    uint16_t temp_AD;
    float temperature;

    // Disable global interrupts during initialization
    DISABLE_INTERRUPTS;

    // System and peripheral initialization
    SystemClock_Config();
    gpio_init(LIGHT_RED, GPIO_OUTPUT, LIGHT_OFF); // Initialize Red LED
    adc_init(AD_TEMP, 0);
    emuart_init(UART_User, 115200);

    // Enable global interrupts after initialization
    ENABLE_INTERRUPTS;

    // UART startup message
    printf("-----------------------------------------------------\n");
    printf("Alarm System Based on Temperature Thresholds
Initialized\n");
    printf("-----------------------------------------------------\n");
```

```
    // Infinite loop
    while (1) {
        // Read temperature from the sensor
        temp_AD = adc_read(AD_TEMP);
        temperature = TempRegression(temp_AD);

        // Handle sensor errors
        if (temperature == -274) {
            printf("Sensor Error: Check sensor connections!\n");
        } else {
            printf("Temperature: %.2f°C\n", temperature);

            // Check if temperature exceeds the threshold
            if (temperature > 20.0) {
                gpio_set(LIGHT_RED, LIGHT_ON); // Turn on Red LED
                printf("ALARM: High Temperature! Red LED ON\n");
            } else {
                gpio_set(LIGHT_RED, LIGHT_OFF); // Turn off Red LED
                printf("Temperature Normal. Red LED OFF\n");
            }
        }

        // Add a delay to stabilize readings
        Delay_ms(1000); // 1-second delay
    }
}

void SystemClock_Config(void) {
    // Add code to configure the system clock as required
}

void Delay_ms(uint16_t u16ms) {
    for (volatile uint32_t i = 0; i < 8000 * u16ms; i++) __ASM("NOP");
}

float TempRegression(uint16_t tmpAD) {
    float Vtemp, Rtemp, temp;

    if (tmpAD <= 72) {
        return -274; // Error value for invalid ADC readings
    }
```

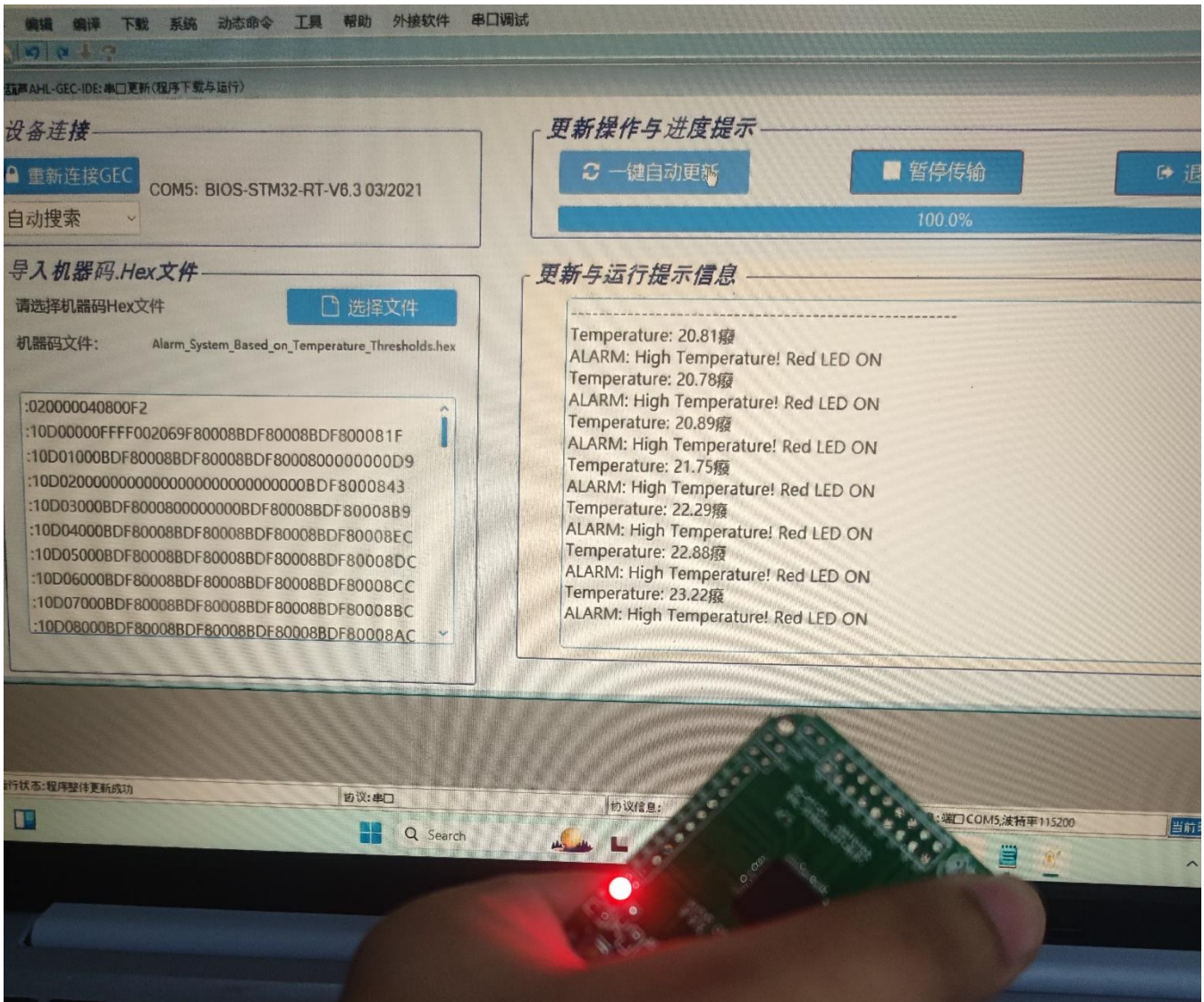
```
    Vtemp = (tmpAD * 3300.0) / 4096; // Convert ADC value to voltage
    Rtemp = Vtemp / (3300.0 - Vtemp) * 10000.0;
    temp = (1 / (log(Rtemp / 10000.0) / 3950.0 + (1 / (273.15 + 25))))
- 273.15 + 0.5;

    return temp; // Return calculated temperature
}
```

Observations and Results

Observations

1. This system was able to ascertain temperature conditions in real time.
2. The red LED was also glowing bright when the temperature reached beyond the threshold level.
3. The UART output was used to present temperature readings and alarm statuses with high accuracy.



Results

Test Case	Condition	Result
Normal Temperature	Temperature ≤ 20°C	Red LED OFF
High-Temperature Alarm	Temperature > 20°C	Red LED ON
Returning to Normal Range	Temperature drops to ≤ 20°C	Red LED OFF

Challenges and Solutions

Challenge 1: Inaccurate Temperature Conversion

- **Issue**: Due to wrong conversion from ADC to temperature false alarms were generated.
- **Solution**: Adjusted the ADC according to the temperature sensor's datasheet.

Challenge 2: Debouncing the LED Alarm

- **Issue**: Owing to temperature variation near the threshold value, the LED is switched on and off frequently.
- **Solution**: Added a time span for temperature readings to settle before raising an alarm.

Conclusion

The **Alarm System Based on Temperature Thresholds** achieved its goal of providing a reliable and responsive method for monitoring temperature conditions and activating visual alarms.

Key outcomes:

1. The system was able to measure temperature through the STM32 ADC peripheral and blinked the red LED when the temperature reached 40°C.

2. It showed a good response time with minimal time delay between the temperature detection and alarm sounding.
3. The system's design in modules and the clearly set levels of the system makes it suitable to be used in various applications that involve temperature measurement.

Future extensions of this project could include:

- Introducing more than one threshold for various levels of alarms such as low, medium, high.
- Adding on the audible alarm that will be in the form of a buzzer.
- Recording the temperatures for future reference in case of an event.
- Adding real time clock (RTC) for time stamped alarm events.

This work presents real-life examples of using embedded systems for safety and environmental management with an opportunity to expand on them for further development.

References

1. STM32L4 Reference Manual
2. Datasheet for STM32L431RC Microcontroller
3. Online Tutorials on GPIO, UART and RTC Configuration